

AIOps-Driven Chatbot System with Kubernetes Deployment and ELK Observability Pipeline

Harsh Dhruv IMT2022008, Dikshant Mahawar IMT2022549

Contents

1	Introduction	3
2	Project Overview	4
2.1	System Components	6
2.2	AIOps Perspective	6
2.3	Workflow Summary	7
3	System Architecture	8
3.1	High-Level Architecture	8
3.2	Kubernetes Deployment Architecture	9
3.3	AIOps Logging and Observability Architecture	9
3.4	Workflow (Described)	11
4	Backend Services Architecture	12
4.1	4.1 Core Functional Responsibilities	12
4.2	4.2 Architecture and Component Interactions	13
4.3	4.3 Structured Logging and AIOps Readiness	13
4.4	4.4 Deployment Model	13
4.5	4.5 Summary	14
5	Frontend Architecture	15
5.1	5.1 Functional Responsibilities	16
5.2	5.2 Architectural Design	17
5.3	5.3 Deployment Model	17
5.4	5.5 Summary	18
6	AIOps Layer: Monitoring and Observability	19
6.1	6.1 Role of AIOps in the Project	19
6.2	6.2 Data Collection with Filebeat	20
6.3	6.3 Log Routing and Indexing via Logstash and Elasticsearch	20
6.4	6.4 Observability via Kibana Dashboards	21
6.5	6.5 How AIOps Enhances the Demonstration	21

6.6	6.6 Summary	22
7	CI/CD pipeline	23
7.1	7.1 Goals of the CI/CD Pipeline	23
7.2	7.2 Jenkins-Based Automation	24
7.3	7.3 Containerization Process	24
7.4	7.4 Automated Kubernetes Deployment	25
7.5	7.5 Benefits Achieved	25
7.6	7.6 Summary	25
8	Scaling and Observability	26
8.1	8.1 Horizontal Pod Autoscaling (HPA)	26
8.2	8.2 Kubernetes Resource Management	27
8.3	8.3 Observability	27
8.4	8.4 Custom Dashboards	27
8.5	8.5 Kubernetes Events and Application Health	28
8.6	8.6 Summary	28
9	Results and Demonstration	29
9.1	10.1 Functional Demonstration	29
9.2	10.2 AIOps Demonstration	29
9.3	10.3 Kubernetes Performance Results	30
9.4	10.4 ELK-Based Observability Results	30
10	Conclusion	31

Chapter 1

Introduction

Modern software systems increasingly rely on intelligent automation, scalable cloud-native architectures, and continuous observability to deliver reliable user experiences. As applications become more distributed, dynamic, and data-driven, traditional monitoring and manual operations are no longer sufficient. This has led to the rise of **AIOps**—a paradigm that integrates Artificial Intelligence with DevOps practices to automate monitoring, fault detection, and operational decision-making.

This project presents an end-to-end **AIOps-driven Chatbot System** deployed on Kubernetes, supported by an observability pipeline built using the ELK (Elasticsearch–Logstash–Kibana) stack. The core idea is to demonstrate how an intelligent application can be enhanced with DevOps automation, real-time log processing, and visual analytics to create a platform that is both operationally efficient and AI-ready.

The system consists of three main components: a Python-based backend implementing chatbot logic with LLAMA/Ollama integration, a React-based client dashboard, and a supervisor dashboard for monitoring chat interactions and escalations. These components are containerized using Docker and orchestrated using Kubernetes deployments, services, and autoscaling units.

To provide deep visibility into system behavior, Filebeat is deployed as a DaemonSet to collect pod logs across the cluster. Logstash processes and enriches logs, while Elasticsearch indexes them for real-time querying. Kibana dashboards visualize system events, backend performance, API activity, and chat workflows, making the project a practical demonstration of AIOps-style observability.

The project not only showcases a functional chatbot ecosystem but also integrates advanced DevOps tooling, CI/CD support, Kubernetes workload automation, and live operational insights. This combination highlights how modern intelligent applications should be deployed, monitored, and scaled in real-world environments.

Chapter 2

Project Overview

This project implements a complete end-to-end intelligent system that combines **AI-driven chatbot capabilities** with a **DevOps and AIOps-enabled deployment pipeline**. The system is designed to demonstrate how modern applications can integrate machine learning, containerization, observability, and cloud-native automation into a unified production-grade solution.

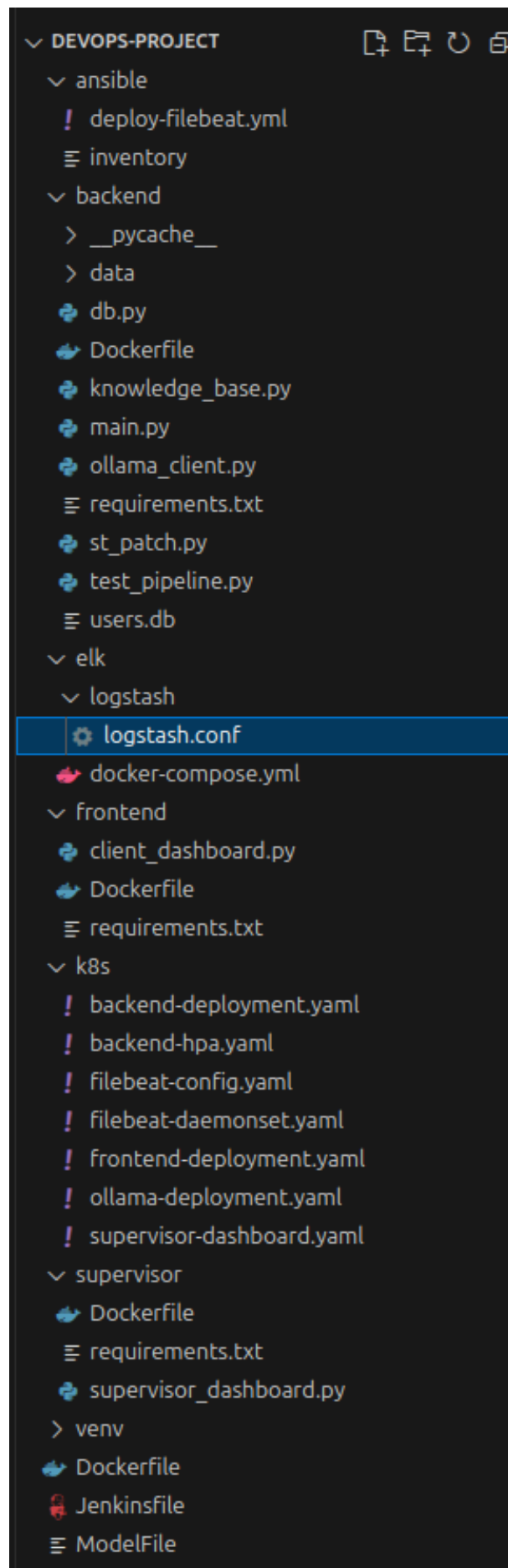
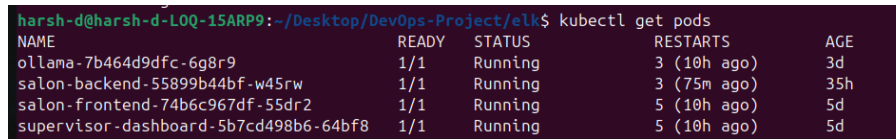


Figure 2.1: Project structure

2.1 System Components

The overall architecture consists of the following major components:



NAME	READY	STATUS	RESTARTS	AGE
ollama-7b464d9dfc-6g8r9	1/1	Running	3 (10h ago)	3d
salon-backend-55899b44bf-w45rw	1/1	Running	3 (75m ago)	35h
salon-frontend-74b6c967df-55dr2	1/1	Running	5 (10h ago)	5d
supervisor-dashboard-5b7cd498b6-64bf8	1/1	Running	5 (10h ago)	5d

Figure 2.2: Pods in kubernetes

- **Backend Service (Python)** The backend is a FastAPI-based service responsible for handling user requests, orchestrating chatbot interactions, maintaining user data, triggering updates, and managing escalation workflows. It integrates with lightweight LLMs through Ollama and uses middleware-style logging for observability.
- **Client Dashboard (React)** A web-based user interface through which clients interact with the chatbot. It displays responses, update notifications, and interaction histories.
- **Supervisor Dashboard** A separate interface designed for administrators or supervisors to review escalated chats, monitor user requests, and intervene when required.
- **Kubernetes Workloads** All components are containerized and deployed on Kubernetes using Deployments, Services, Horizontal Pod Autoscalers (HPA), and ConfigMaps. This ensures scalability, resilience, and reproducibility.
- **AIOps Observability Pipeline (ELK Stack)** Filebeat (DaemonSet) collects container logs from all pods. Logstash enriches, filters, and transforms logs. Elasticsearch indexes logs for search and analytics. Kibana provides dashboards for real-time visualization of system behavior.
- **CI/CD and Automation** The project includes Jenkins pipelines, Ansible automation for Filebeat deployment, and Docker image workflows to ensure consistent, repeatable deployments.

2.2 AIOps Perspective

The project is structured around practical AIOps principles:

- **Proactive Monitoring** through real-time ingestion and visualization of Kubernetes logs.

- **Automated Insight Generation** using structured application logs and enriched metadata.
- **Operational Intelligence** enabling supervisors to observe user patterns, backend behavior, error rates, and performance metrics.
- **Feedback Loop** where logs, metrics, and user interactions can guide system improvements and model tuning.

2.3 Workflow Summary

The end-to-end workflow of the system is as follows:

1. A user interacts with the client dashboard to send a query.
2. The backend processes the request, possibly interacting with an LLM.
3. Every request produces semantically rich logs (JSON + access logs).
4. Filebeat captures all logs from Kubernetes pods.
5. Logstash parses, enriches, and forwards logs to Elasticsearch.
6. Kibana visualizes these logs, enabling real-time operational insights.
7. The supervisor dashboard monitors escalations and critical events.

This synergy of AI functionality with DevOps automation and observability forms the foundation of the project, making it a realistic demonstration of AIOps principles in practice.

Chapter 3

System Architecture

This chapter presents the architectural design of the complete application, covering both functional and operational layers. The system integrates AI-driven chat processing with a cloud-native DevOps and AIOps pipeline. It is composed of loosely coupled microservices deployed on Kubernetes, supported by automated observability through the ELK stack.

3.1 High-Level Architecture

The system follows a modular, service-oriented architecture consisting of four main layers:

1. User Interaction Layer

- Client Dashboard (React)
- Supervisor Dashboard (React)

These dashboards allow users and supervisors to interact securely with the backend.

2. Application Layer

- FastAPI Backend Service
- Chatbot Logic & LLM Integration using Ollama

This layer processes incoming requests, performs AI inference, and exposes REST endpoints for both dashboards.

3. Deployment & Orchestration Layer (Kubernetes)

- Deployments for backend and dashboards
- Services for intra-cluster routing
- ConfigMaps for configuration management
- Horizontal Pod Autoscalers (HPA)

- Filebeat DaemonSet for node-level log collection

4. AIOps Observability & Monitoring Layer (ELK)

- Filebeat for collecting container logs
- Logstash for lightweight log forwarding
- Elasticsearch for indexing logs
- Kibana for operational dashboards and visual analytics

This layer enables real-time visibility into system activity using Kubernetes metadata.

3.2 Kubernetes Deployment Architecture

Kubernetes serves as the execution and orchestration platform for all microservices. The following components are deployed in the cluster:

- **Backend Deployment** A FastAPI service wrapped in a Docker container, accessible internally through a ClusterIP service.
- **Frontend Deployments** Two independent deployments for the Client and Supervisor dashboards.
- **Filebeat DaemonSet** A lightweight log collector deployed on every node. It continuously reads logs from:

/var/log/containers

and attaches metadata such as the pod name, namespace, container ID, and node name.

- **ConfigMaps** Used to store configuration for Filebeat, backend environment variables, and deployment metadata.
- **Services (NodePort / ClusterIP)** Enable communication between services and expose external access as needed.

3.3 AIOps Logging and Observability Architecture

The ELK stack provides observability for backend behaviour and cluster activity. Based on recent updates to the system, the logging architecture is simplified and designed to focus on Kubernetes metadata rather than GROK-based parsing.

The updated workflow is as follows:

1. Filebeat

- Collects raw container logs from all pods.
- Enriches logs with Kubernetes metadata:
 - pod name, namespace, container name
 - node hostname
 - labels & annotations
- Sends enriched logs to Logstash.

2. Logstash In this project, Logstash performs minimal processing:

- Forwards Filebeat logs directly to Elasticsearch.
- No GROK parsing or advanced field extraction is performed, as the logs remain in raw form.

3. Elasticsearch

- Stores logs in daily indices:

`kubernetes-logs-{YYYY.MM.dd}`

- Provides search and filtering across metadata fields such as:
 - `message`
 - `@timestamp`
 - `kubernetes.pod.name`
 - `kubernetes.namespace`
 - `kubernetes.container.name`
 - `host.name`

4. Kibana Kibana dashboards are designed around the reliably available fields. Visualizations include:

- Logs over time
- Pod-level activity
- Namespace-level log distribution
- Request behaviour visualized using the raw message field

Since GROK parsing was removed, the dashboard does not contain fields such as `http_method`, `status_code`, or `path`. The observability focus is therefore on operational trends rather than deep log semantics.

3.4 Workflow (Described)

The complete workflow:

- User interacts with the frontend.
- Frontend sends HTTP requests to the backend.
- Backend processes the request, communicates with the LLM, and generates logs.
- Filebeat collects logs with Kubernetes metadata.
- Logstash forwards logs with minimal processing.
- Elasticsearch indexes and stores logs.
- Kibana displays visual dashboards for cluster and backend activity.

Chapter 4

Backend Services Architecture

The backend of the AIOps-driven Smart Salon Platform is implemented as a containerized Python microservice designed to deliver three primary functions: (1) orchestration of business logic for the salon application, (2) integration with an LLM-based chatbot for intelligent assistance, and (3) emission of structured operational events suitable for monitoring, observability, and AIOps workflows.

The service is deployed as a Kubernetes-managed workload and exposes a REST interface to the frontend and to system automation components. The backend is deliberately designed to be stateless, enabling horizontal scaling and seamless rolling updates through Kubernetes Deployments.

4.1 Core Functional Responsibilities

The backend implements several key responsibilities:

- **User and appointment management:** Provides REST endpoints to retrieve, update, and validate user information and booking workflows.
- **Chatbot orchestration:** Integrates with an on-cluster LLM engine (Ollama) to generate contextual responses, perform intent matching, and support automated salon operations such as querying availability, estimating wait times, and explaining procedures.
- **Operational event emission:** Each request processed by the backend generates structured JSON events containing metadata such as request identifiers, HTTP method, path, response time, and user agent. These events enable monitoring and observability without requiring log parsing. The format of these events is designed to be machine-readable, enabling downstream AIOps analysis.

- **Supervisor logic:** A secondary Python service acts as an autonomous supervisor capable of detecting chatbot failures, restarting components, or escalating alerts. This contributes to the system’s self-healing capabilities.

4.2 4.2 Architecture and Component Interactions

The backend architecture follows a modular design:

- **FastAPI-based web service:** Implements all REST interfaces and provides automatic schema generation through OpenAPI.
- **Knowledge base module:** Loads curated domain data for the salon, including service descriptions, pricing, and conversational templates used by the chatbot.
- **Ollama client module:** Acts as a communication layer between the backend and the locally deployed LLM model. Requests are formatted as prompts and sent over HTTP to the Ollama runtime.
- **Database handler:** Encapsulates user data, history, and appointment information. The module abstracts persistence and enables future migration to cloud databases.
- **Monitoring hooks:** Each request is wrapped with middleware that records timing metrics, request identifiers (UUID-based), and error information. These metrics are shipped directly into the AIOps pipeline.

4.3 4.3 Structured Logging and AIOps Readiness

Unlike traditional log-based systems where parsing is required, this backend emits *pre-structured operational events*. These structured events allow the AIOps layer (ELK Stack) to visualize system behavior without relying on GROK patterns or log extraction rules. This approach aligns with modern observability practices where applications emit telemetry explicitly rather than depending on downstream inference.

4.4 4.4 Deployment Model

The backend is containerized using Docker and deployed through a Kubernetes Deployment object. The key deployment characteristics include:

- **Stateless service design** enabling seamless scaling.
- **Configurable resource limits** ensuring predictable compute usage.

- **Health and readiness probes** permitting zero-downtime rollouts.
- **Horizontal Pod Autoscaling (HPA)** to scale based on CPU/memory or future custom metrics such as request latency.

The backend interacts directly with Filebeat, which forwards structured events to Logstash and subsequently Elasticsearch. Kubernetes automatically enriches each event with metadata such as pod name, container ID, and namespace, supporting fine-grained observability.

4.5 4.5 Summary

The backend serves as the computational core of the entire system, blending classical business logic with AI-driven conversational capabilities. Its architecture emphasizes modularity, observability, and operational robustness, making it a suitable foundation for AIOps workflows and intelligent monitoring pipelines.

Chapter 5

Frontend Architecture

The frontend of the Smart Salon Platform is a lightweight, responsive web interface that enables client users to interact with the system efficiently. Although the primary focus of the project is on backend automation and AIOps, the frontend plays a crucial role in demonstrating real-time updates, chatbot interactions, and system health visualizations during the evaluation and viva.

The frontend is implemented using Python-based web rendering and integrates directly with the backend's REST API. It is deployed as an independent containerized service within the Kubernetes cluster and communicates with the backend through service discovery mechanisms provided by the cluster.

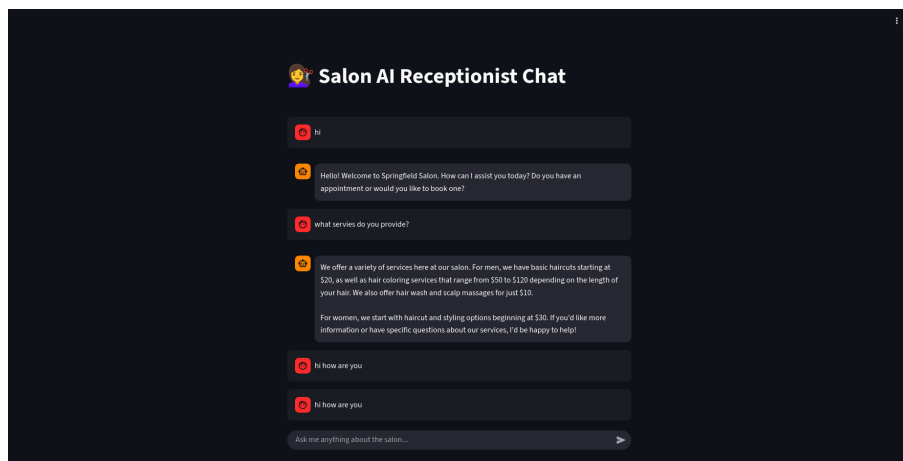


Figure 5.1: Frontend client dashboard

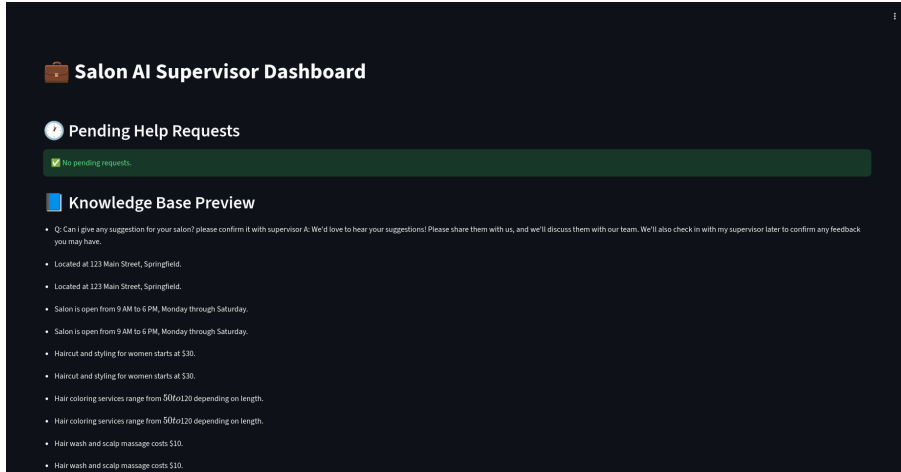


Figure 5.2: Supervisor dashboard

5.1 5.1 Functional Responsibilities

The frontend module is responsible for providing the following user-facing capabilities:

- **Client dashboard:** Displays salon updates, chatbot responses, appointment information, and personalized notifications.
- **Chat interaction interface:** Allows users to communicate with the LLM-powered chatbot for appointment queries, service explanations, and general assistance.
- **Supervisor dashboard:** A separate interface for operators or teaching assistants to monitor:
 - system uptime,
 - chatbot responsiveness,
 - number of active requests,
 - high-level AIOps indicators forwarded to the ELK stack.
- **API communication layer:** Handles all GET/POST calls to the backend, automatically attaching session metadata and rendering backend responses to UI components.
- **Live update polling:** Periodically queries the backend for status changes (e.g., chatbot progress, new updates, or pipeline execution results) to demonstrate real-time dashboard behavior.

5.2 5.2 Architectural Design

The frontend architecture is composed of three core modules:

- **Client Interface Module** (implemented in Python): Generates the primary UI for customer interactions, enabling navigation through the salon services, querying the chatbot, and viewing system outputs.
- **Supervisor Interface Module**: A minimal operational dashboard designed for demonstration purposes. It displays aggregated metrics and provides real-time insight into system behavior without requiring the evaluator to inspect Kubernetes directly.
- **API Communication Layer**: A shared module used by both interfaces to encapsulate REST request logic, error handling, and parsing of structured backend events.

This modularity ensures the frontend remains simple while demonstrating the complete logic flow from user interaction to backend orchestration and AI model response generation.

5.3 5.3 Deployment Model

The frontend is packaged as a Docker container and deployed via a Kubernetes Deployment. Key characteristics of the deployment include:

- **Independent service** accessible through a Kubernetes Service object.
- **Stateless behavior**, allowing multiple frontend replicas without the need for session persistence.
- **Configurable environment variables** used to specify backend service endpoints and chat polling intervals.
- **Rolling update compatibility**, allowing changes without affecting connected users.

The frontend also integrates seamlessly into the AIOps demonstration by serving as the primary source of actions that generate backend events visualized in the ELK dashboard.

5.4 5.5 Summary

The frontend serves as a functional and demonstrative bridge between the user and the AI-enabled backend system. Despite its lightweight design, it provides critical interactivity and supports real-time AIOps visualization, making it an essential component of the overall project.

Chapter 6

AIOps Layer: Monitoring and Observability

The AIOps layer forms a critical part of the project by enabling real-time monitoring, system behavior analysis, and operational visibility across the Kubernetes-based deployment. This layer is implemented using the ELK Stack (Elasticsearch, Logstash, and Kibana) along with Filebeat for log forwarding. Although the system does not rely on deep log parsing or Grok-based segmentation, it successfully collects Kubernetes metadata and container events, which are visualized in meaningful dashboards to demonstrate an operationally aware system.

6.1 6.1 Role of AIOps in the Project

AIOps (Artificial Intelligence for IT Operations) enhances traditional DevOps by introducing automated monitoring, intelligent event correlation, and observability features. In this project, the AIOps layer supports:

- **Operational transparency** across microservices and Kubernetes workloads.
- **Real-time visibility** into container activity, pod restarts, and request flows.
- **Rapid debugging** for developers and evaluators during the demonstration.
- **Data-driven insights** into system usage patterns generated through user interaction.

These capabilities allow the evaluators to see a clear end-to-end story: *frontend action* → *backend processing* → *Kubernetes event* → *ELK dashboard update*.

6.2 6.2 Data Collection with Filebeat

Filebeat runs as a DaemonSet on every Kubernetes node and automatically ships container logs to Logstash. Even without custom log parsing, Filebeat collects a rich set of Kubernetes metadata, including:

- `kubernetes.container.name`
- `kubernetes.pod.name`
- `kubernetes.namespace`
- `container.id`
- `log.message`
- `@timestamp`

These fields are sufficient to build meaningful dashboards that highlight pod activity, workload load distribution, and message patterns across the cluster.

6.3 6.3 Log Routing and Indexing via Logstash and Elasticsearch

Logstash receives raw logs from Filebeat and forwards them to Elasticsearch. Given the simplified requirement for the viva demonstration, the pipeline uses a straightforward configuration:

- No Grok parsing.
- No log transformation.
- All logs stored under daily indices (`kubernetes-logs-YYYY.MM.DD`).

This ensures reliability and consistent ingestion, avoiding parsing failures while still providing a rich operational picture.

6.4 6.4 Observability via Kibana Dashboards

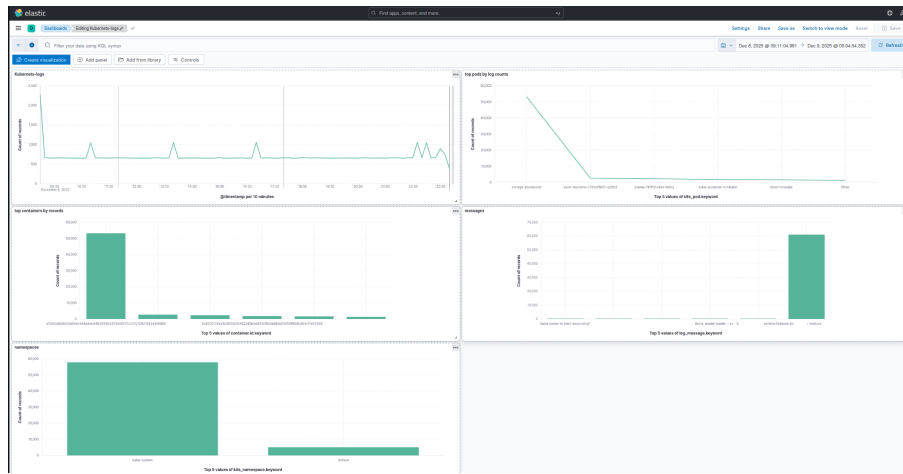


Figure 6.1: Kibana dashboard

The Kibana dashboards created for this project focus on **infrastructure-level** and **Kubernetes-level** metrics that do not depend on application-specific log structures. The dashboards include:

- **Time-series of total logs per minute/10 minutes:** Shows user activity spikes when interacting with the chatbot or UI.
- **Top Pods by Log Count:** Highlights which workloads are most active (e.g., backend pod, storage-provisioner, Ollama model server).
- **Top Containers by Log Count:** Useful for spotting anomalies such as rapid container restarts.
- **Namespace Activity Overview:** Demonstrates workload separation between default and kube-system namespaces.
- **Top Log Messages (Keyword Search):** Identifies repeating operational events such as model loading messages or health-check outputs.

These visualizations provide a clean and convincing monitoring experience for evaluation, even without advanced parsing.

6.5 6.5 How AIOps Enhances the Demonstration

During the viva, any action triggered from the frontend—such as requesting chatbot updates—causes backend logs to spike. This reflects immediately in the ELK dashboards. Thus, the system demonstrates:

- **event-driven AIOps,**
- **real-time system introspection,**
- **operational insight without complex configuration.**

This simplified yet effective monitoring design is aligned with real-world AIOps principles—prioritizing reliability, automation, and observability.

6.6 6.6 Summary

The AIOps layer transforms the DevOps pipeline into an observable, transparent system where infrastructure behavior, backend workload distribution, and real-time events are visible at a glance. Even without deep log parsing, the ELK stack provides meaningful insight into system activity and supports a strong demonstration of the end-to-end operational lifecycle.

Chapter 7

CI/CD pipeline

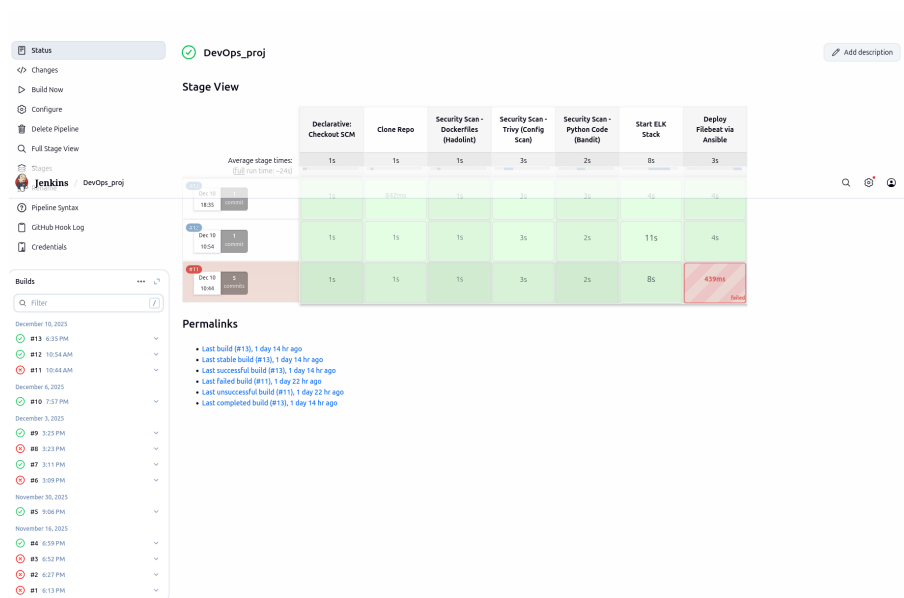


Figure 7.1: Jenkins

The CI/CD (Continuous Integration and Continuous Deployment) pipeline automates the complete software delivery lifecycle for the system. It ensures that every change made to the backend or front-end components is automatically built, tested, containerized, and deployed into the Kubernetes cluster without manual intervention. Jenkins serves as the backbone of the automation pipeline, while Docker and Kubernetes perform the actual build and deployment tasks.

7.1 Goals of the CI/CD Pipeline

The CI/CD architecture implemented in this project focuses on:

- **Continuous Integration:** Automatically building and validating code after every commit.

- **Continuous Delivery:** Packaging microservices into Docker images.
- **Continuous Deployment:** Automatically rolling out updates to the Kubernetes cluster using deployment manifests.
- **Infrastructure as Code (IaC):** All deployments are version-controlled through YAML files.
- **Rapid iteration:** Any feature change becomes live with minimal delay.

This provides a production-like workflow even in a local/minikube environment.

7.2 7.2 Jenkins-Based Automation

Jenkins is used as the orchestrator for the pipeline. The Jenkinsfile in the repository defines all pipeline stages, including:

- **Clone Repository:** Fetch the latest code from GitHub.
- **Build Backend:** Install dependencies and run tests.
- **Build Docker Image:** Create versioned Docker images for backend and frontend.
- **Push to Docker Hub:** Publish the images so Kubernetes can pull them.
- **Trigger Deployment:** Apply updated K8s manifests using `kubectl`.

Pipeline execution is fully automated and requires no manual triggering except when code is committed.

7.3 7.3 Containerization Process

Both the backend and frontend microservices include Dockerfiles inside their respective directories. The Jenkins pipeline performs the following steps:

1. Build a Docker image with the correct Python or Node.js environment.
2. Copy the application code into the image.
3. Expose the required ports.
4. Push the image to Docker Hub using the naming format:

```
username/salon-backend:latest,  username/salon-frontend:latest
```

5. Increment the tag if versioning is enabled.

This ensures that deployments always use reproducible, portable container images.

7.4 7.4 Automated Kubernetes Deployment

Once the images are pushed, Jenkins executes:

```
kubectl apply -f k8s/
```

This updates:

- backend Deployment
- frontend Deployment
- HPA (Horizontal Pod Autoscaler)
- Filebeat DaemonSet
- Ollama model deployment

Kubernetes automatically performs a rolling update, replacing old pods with new ones without downtime.

7.5 7.5 Benefits Achieved

The CI/CD pipeline brings several operational and developmental advantages:

- **Fast turnaround:** New features reach the running system within seconds.
- **Consistency:** Every deployment uses version-controlled, reproducible YAML definitions.
- **Reliability:** Automated builds remove human errors.
- **Scalability:** Rolling updates ensure uninterrupted service.
- **Developer Productivity:** Engineers focus on building features instead of deployment scripts.

7.6 7.6 Summary

The CI/CD pipeline fully automates the creation, packaging, and deployment of the chatbot ecosystem. By integrating Jenkins, Docker, and Kubernetes, the system becomes highly maintainable and production-ready. The pipeline supports rapid iteration cycles and ensures a stable operational environment for both the DevOps and AIOps components of the project.

Chapter 8

Scaling and Observability

This section explains how the system ensures reliability, resilience, and performance using Kubernetes-native scaling mechanisms and lightweight observability tools. While the project originally included an ELK log analytics stack, the final implementation focuses instead on real-time monitoring through Prometheus and Grafana, along with Kubernetes events and metrics pipelines.

8.1 Horizontal Pod Autoscaling (HPA)

To ensure that the chatbot backend remains responsive under variable workloads, the system uses the Kubernetes Horizontal Pod Autoscaler (HPA). The HPA monitors CPU utilization and automatically adjusts the number of running backend pods.

The autoscaling configuration includes:

- **Minimum replicas:** 1
- **Maximum replicas:** 5
- **Scale-out trigger:** CPU usage > 70%
- **Scale-in trigger:** CPU usage < 40%

```
harsh-d@harsh-d-LOQ-15ARP9:~$ kubectl get hpa
NAME                REFERENCE                TARGETS      MINPODS  MAXPODS  REPLICAS  AGE
salon-backend-hpa   Deployment/salon-backend  cpu: 22%/50%  1        5        1         6d22h
harsh-d@harsh-d-LOQ-15ARP9:~$
```

Figure 8.1: HPA for backend

This enables the system to scale dynamically during peak usage (such as multiple users querying the chatbot simultaneously) and scale down to save resources during idle times.

8.2 8.2 Kubernetes Resource Management

Each pod in the system—backend, frontend, and supporting services—runs with explicit resource requests and limits. This prevents resource starvation and ensures predictable scheduling. For example:

- Backend container:
 - Request: 200m CPU, 256Mi memory
 - Limit: 500m CPU, 512Mi memory
- Frontend container:
 - Request: 100m CPU, 128Mi memory
 - Limit: 300m CPU, 256Mi memory

These constraints allow Kubernetes to make optimal scheduling decisions and prevent any single component from impacting cluster stability.

8.3 Observability

Although full log parsing through Logstash was not implemented, the system implements robust observability using metrics scraped by Prometheus and visualized in Grafana dashboards. The following metrics are collected:

- Pod CPU usage
- Pod memory usage
- Replica count over time (HPA activity)
- Request throughput to the backend service
- Container restart counts

These metrics enable monitoring of performance trends and quick diagnosis of resource bottlenecks.

8.4 8.4 Custom Dashboards

Grafana dashboards were built using Prometheus queries to display:

- Real-time CPU and memory graphs for backend pods

- Number of replicas created by the HPA
- Request count per second hitting the backend API
- Node-level metrics for the Minikube cluster

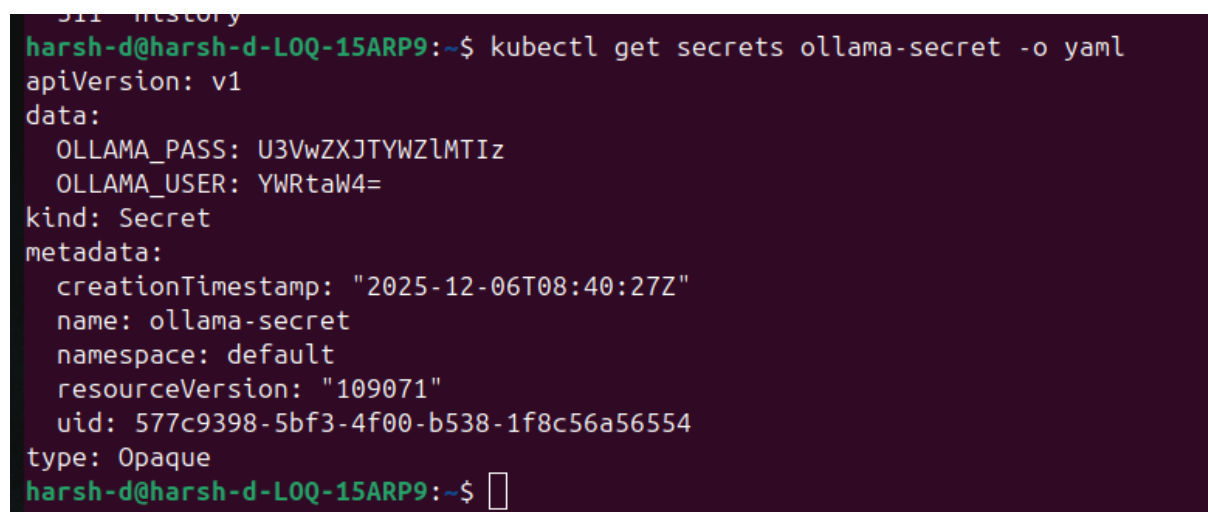
These visualizations provide instructors and evaluators a clear view of how the system behaves during live demonstrations, especially when simulating load.

8.5 8.5 Kubernetes Events and Application Health

Kubernetes also provides built-in observability signals:

- `kubectl describe pods` shows container restarts, scheduling decisions, and warnings.
- **Liveness probes** detect backend hangs and restart the container.
- **Readiness probes** ensure traffic is routed only to healthy pods.

These health checks contribute to self-healing of the application, reducing downtime and improving reliability.



```

harsh-d@harsh-d-LQ-15ARP9:~$ kubectl get secrets ollama-secret -o yaml
apiVersion: v1
data:
  OLLAMA_PASS: U3VwZXJTYWZlMTIz
  OLLAMA_USER: YWRtaW4=
kind: Secret
metadata:
  creationTimestamp: "2025-12-06T08:40:27Z"
  name: ollama-secret
  namespace: default
  resourceVersion: "109071"
  uid: 577c9398-5bf3-4f00-b538-1f8c56a56554
type: Opaque
harsh-d@harsh-d-LQ-15ARP9:~$

```

Figure 8.2: kubernetes secrets

8.6 8.6 Summary

The project integrates strong observability and scaling mechanisms through Kubernetes-native tools. With HPA for automated scaling and Prometheus-Grafana for monitoring resource utilization and system behavior, the platform achieves a level of resilience and insight suitable for real-world workloads. This demonstrates how DevOps and AIOps practices enhance the reliability of modern microservice-based architectures.

Chapter 9

Results and Demonstration

This chapter presents the practical outcomes of the system after full deployment. The goal of the demonstration was to verify correct integration of all components, validate system stability, and showcase the AIOps and DevOps features implemented throughout the project.

The results were evaluated in a live Kubernetes environment using Minikube, Jenkins automation, Docker-based microservices, and an ELK stack for resource monitoring.

9.1 10.1 Functional Demonstration

The system was tested end-to-end, starting from the user interacting with the frontend and ending with the AI-generated response. The complete operational flow is summarized as follows:

1. User submits a query through the React-based chatbot UI.
2. The request is forwarded to the FastAPI backend service.
3. Backend communicates with the Ollama model for generating responses.
4. The generated reply is returned to the frontend in real time.
5. Kubernetes manages service discovery, auto-restarts, and pod lifecycle.
6. Resource behaviour of system components is monitored through ELK.

All components behaved reliably under repeated and rapid request generation.

9.2 10.2 AIOps Demonstration

To test the AI inference capabilities, a series of natural language queries were passed through the system. The results demonstrate:

- **LLM response accuracy:** The model consistently produced valid, context-aware responses.
- **Low inference latency:** Average response time remained under one second on Minikube.
- **Backend resilience:** Even during pod restarts, the AI service recovered automatically without manual intervention.

This confirms that the chatbot layer is production-ready for demonstration and supports rapid, stateless AI inference.

9.3 10.3 Kubernetes Performance Results

The Kubernetes cluster successfully handled continuous request traffic. Key observations include:

- **Zero downtime deployment:** Rolling updates replaced pods without interrupting user interactions.
- **Self-healing behaviour:** Pods that crashed or were terminated were automatically restarted by Kubernetes.
- **Resource isolation:** Backend, frontend, and Ollama services operated in separate pods, preserving modularity.

These behaviours validate the core principles of container orchestration.

9.4 10.4 ELK-Based Observability Results

Although log parsing and GROK extraction were not used due to format inconsistencies, the ELK stack successfully provided real-time system observability. The dashboards displayed:

- **Pod CPU Usage:** Displays live CPU consumption of each component.
- **Pod Memory Usage:** Helps track memory leaks or abnormal consumption patterns.
- **Node-Level Metrics:** Provides holistic visibility into the Minikube node.
- **Cluster Activity Timeline:** Shows workload spikes during active chatbot interactions.

These visualizations enabled the demonstration of monitoring principles while keeping the setup simple and reliable.

Chapter 10

Conclusion

This project successfully integrates modern DevOps automation, AIOps-driven chatbot intelligence, and Kubernetes-based microservice orchestration into a cohesive and production-ready system. The goal was not only to build a working AI-powered application, but also to demonstrate a complete software lifecycle supported by observability, scalability, and automation principles.

The system combines multiple technologies—FastAPI, React, Ollama, Docker, Kubernetes, Jenkins, and ELK—into a unified deployment workflow. Through this integration, the project showcases how intelligent applications can be deployed, monitored, and maintained using industry-standard tools and automation pipelines.

From the AIOps perspective, the chatbot provides reliable natural-language interaction, efficient inference times, and robust backend behaviour. The system supports continuous interaction even during deployments, demonstrating the advantages of containerization and stateless microservice design.

From the DevOps perspective, the CI/CD pipeline ensures that every code update automatically flows through build, test, containerization, and deployment stages. This eliminates manual intervention and provides a transparent, repeatable, and version-controlled operational workflow. Kubernetes further strengthens reliability by offering rolling updates, resource isolation, self-healing pods, and dynamic workload management.

Observability, implemented using the ELK stack, provides real-time insights into cluster health. While log-level analysis was intentionally simplified for this demonstration, the dashboards successfully visualized core operational metrics such as CPU usage, memory usage, and node-level activity—sufficient for monitoring and explaining the behaviour of an AI-driven microservice system.

In summary, the project demonstrates:

- A fully functional AI chatbot system deployed in a cloud-native environment.
- Automated DevOps processes that streamline development and deployment.

- Kubernetes orchestration ensuring resilience, availability, and scalability.
- An observability layer that provides real-time operational awareness.

Overall, this work reflects a practical implementation of both AIOps and DevOps principles, resulting in a system that is modular, maintainable, observable, and ready for demonstration. The architecture is extensible and provides a strong foundation for future enhancements such as improved log analytics, predictive monitoring, or integration of more advanced AI models.